

PlantCare Web Application

Comprehensive Project Documentation & System Reference Manual

1. Project Overview & Architecture

PlantCare is a smart plant health diagnostic and agricultural advisory web application. It matches user-submitted crop scans to localized crop disease libraries, evaluates real-time crop growth indexes, and records scan details into a specialized machine learning database. The project is structured with a Django REST backend, React frontend (Axios/JWT/Recharts), and is integrated with the Pl@ntNet API and OpenWeatherMap APIs.

2. Backend Modules & Database Schema

A. Accounts Module (accounts/)

Extends Django's AbstractUser to store user accounts, physical farm profiles (size, crop settings, name), theme configurations (light/dark), and translation catalogs (English, Hindi, Gujarati).

B. Library Reference Module (library/)

Serves as the central dictionary catalog. Contains Crops (ideal temperature, optimal humidity ranges), Diseases (causes, symptoms, treatment guidelines), and Fertilizer recommendations, fully translated.

C. Scans Module (scans/ & plantcare/)

Manages diagnostic crop scan uploads. Resolves plant species, matches against local library diseases, and saves the history of scans with resolved locations.

D. Weather Module (weather/)

Integrates OpenWeatherMap current forecasts, past history meteorological data, and crop growth chance ratings.

3. Local Environment Setup Guide

Step 1: Create a Python Virtual Environment

```
python -m venv .venv
```

Step 2: Activate the Virtual Environment

On Windows (PowerShell):

```
.venv\Scripts\Activate.ps1
```

On macOS / Linux:

```
source .venv/bin/activate
```

Step 3: Install Required Packages

```
pip install -r requirements.txt
```

Step 4: Configure API Credentials (.env file)

Generate a .env file in the root directory and specify your key credentials:

- PLANTNET_API_KEY: Your Pl@ntNet key (my.plantnet.org)
- OPENWEATHER_API_KEY: Your OpenWeather key (openweathermap.org)

```
copy .env.example .env
```

Step 5: Run Migrations and Seed Data

Create both default and FUTUREDATASET tables, and seed initial crops and admin credentials:

```
python manage.py migrate --database=default
python manage.py migrate --database=FUTUREDATASET
python manage.py seed_data
```

Step 6: Launch Server

```
python manage.py runserver
```

4. Recent System Upgrades & Multi-Database ML Logging

A. Automatic Environment Variables Auto-Loading

Environment configurations inside the `.env` file are automatically loaded when running any commands or running the development server via `python-dotenv` integration at the startup entry points of `manage.py`, `plantcare/wsgi.py`, and `plantcare/asgi.py`. Manual exports in terminal are not required.

B. Full-Word Language Toggle & Sidebar Stacking

The language selectors now show the full localized names: 'English', 'Hindi', and 'Gujarati'. To accommodate these names without sidebar layout breaking on narrow screen widths, the language selection buttons and the visual theme toggle forms are stacked vertically.

C. Unified Location Resolution & Geocoding Fallback Flow

Both Weather Advisor and Plant Health Scan features resolve location according to this priority order:

1. User Profile City: Geocodes `request.user.location_city` via OpenWeather Geocoding API.
2. User Profile GPS: Falls back to user's saved profile coordinates (latitude, longitude).
3. Browser Geolocation: Prompts browser's `navigator.geolocation` via frontend script if no location is configured.

D. Dedicated ML FUTUREDATASET Database Logging

A secondary database configuration key 'FUTUREDATASET' pointing to `FUTUREDATASET.db` (SQLite) has been introduced. This database stores a structured dataset table named 'FUTUREDATASET' mapping to the `FutureDataset` model. Every successful scan upload (via Web UI and REST API) automatically duplicates the scan attributes and prediction outputs (species, disease, healthy/diseased, coordinates) in `FUTUREDATASET.db` to build a structured training dataset for future machine learning and deep learning models.

E. User Account Deletion Data Retention

The `FutureDataset` model in `FUTUREDATASET` database logs the uploader's username as a plain `CharField` string instead of using a `ForeignKey` to the Django user database table.

When user accounts are deleted: the default database cascades and deletes the user's history from the application. However, the accumulated training logs in the `FUTUREDATASET` database remain completely intact, ensuring valuable model training data is never lost.